

UNIT 2: PROCESS DESCRIPTION AND CONTROL

Course Pattern: TE IT - 2024 Pattern (SPPU)	Academic Term: Semester V (2026)
Status: Publication-Grade Theory Guide	Course Code: PCC-301-ITT

Syllabus Mapping & Unit Structure

The following table outlines the curriculum mapping and priority weightage for Unit 2.

Syllabus Topic	In-Depth Exam Coverage	Priority / Weightage
Process Concept & States	Process memory address space (Text, Data, Heap, Stack), 5-State model, 7-State model with swapping mechanics, PCB structure & attributes.	High Priority (8-10 Marks Qs)
Process Control & POSIX API	OS execution models, Mode Switch vs Context Switch overhead, POSIX process management (fork, exec, wait, exit), Zombie vs Orphan processes.	High Priority (8-10 Marks Qs)
Threads & Multithreading	Processes vs Threads, Single vs Multithreaded architecture, User-Level Threads (ULT) vs Kernel-Level Threads (KLT), Mapping models (1:1, M:1, M:N), Pthreads.	High Priority (6-8 Marks Qs)
CPU Scheduling Principles	Types of Schedulers (Long/Medium/Short-term), Scheduling criteria metrics, Preemptive vs Non-Preemptive algorithms (FCFS, SJF, SRTF, Priority, RR, MLQ, MLFQ).	High Priority (8-10 Marks Qs)

Process Description and Process Control Block

Process Concept and Address Space Architecture

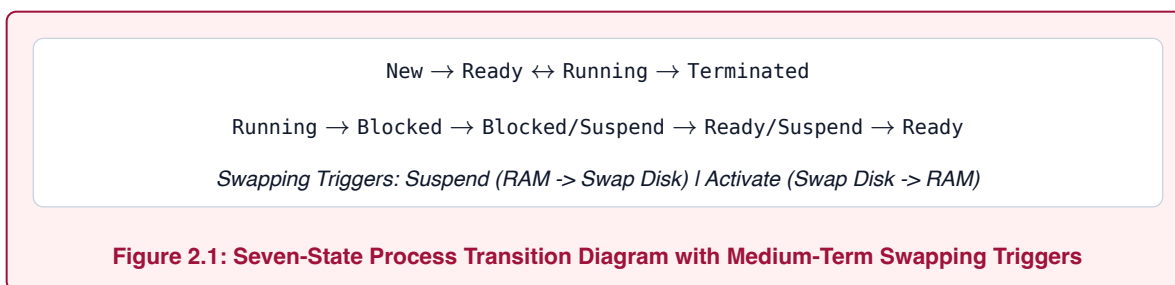
A **Process** is defined as a program in execution. While a program is a passive entity stored on disk (executable file), a process is an active entity possessing an assigned processor counter, set of CPU registers, allocated system resources, and an isolated virtual address space.

- Text Segment (Code Segment):** Contains executable binary machine instructions read from disk. Usually marked read-only to permit safe sharing across concurrent instances.
- Data Segment:**
 - Initialized Data:** Stores global and static variables explicitly initialized by the programmer.
 - Uninitialized Data (BSS):** Stores global and static variables initialized to zero by default at process startup.
- Heap Segment:** Dynamically allocated memory space managed at runtime via memory calls (malloc(), calloc(), new, free()). Grows upwards toward higher memory addresses.
- Stack Segment:** Stores temporary stack frames for active function calls (local variables, function arguments, return addresses, saved register states). Grows downwards toward lower memory addresses.

Process State Transitions: 5-State vs. 7-State Model

- Five-State Process Model:**

- **New:** Process is being created by the OS but not yet loaded into main memory ready queue.
 - **Ready:** Process is resident in RAM, fully prepared to execute, awaiting CPU assignment.
 - **Running:** Instructions are actively being executed by a CPU core.
 - **Blocked (Waiting):** Process cannot execute until an external event or I/O operation completes.
 - **Terminated (Exit):** Process has halted execution; resources are reclaimed by the OS.
2. **Seven-State Process Model (Swapping Mechanics):** When main memory becomes overcommitted, the OS **Medium-Term Scheduler** swaps suspended processes out of RAM to secondary storage (swap disk area), introducing two suspended states:
- **Ready / Suspend:** Process is in secondary storage but ready to execute as soon as it is swapped back into RAM.
 - **Blocked / Suspend:** Process is in secondary storage and waiting for an I/O event.



Structure and Attributes of Process Control Block (PCB)

The **Process Control Block (PCB)** is the central data structure maintained in kernel memory to represent and control each active process.

1. **Process Identification:** Unique **Process ID (PID)**, Parent PID (PPID), User ID (UID), and Group ID (GID).
2. **Processor State Information:** Program Counter (PC), CPU data registers, Stack Pointer (SP), Index registers, and Condition Code flags.
3. **Process Control Information:**
 - **Process State:** Current status (Ready, Running, Blocked, Suspended).
 - **Scheduling Priority:** Priority level, CPU usage metrics, scheduling queue pointers.
 - **Memory Management:** Base and limit registers, Page Table pointers, or Segment Table references.
 - **I/O & Accounting:** List of open file descriptors, allocated I/O devices, total CPU time consumed, memory quotas.

Source: Stallings, Ch 3, p. 105-130 | Source: Silberschatz, Ch 3, p. 105-125

Process Control & POSIX API

Mode Switch vs. Context Switch

Parameter	Mode Switch	Context Switch
Definition	Transition of CPU hardware execution privilege between User Mode (1) and Kernel Mode (0).	Saving execution context of current process and restoring saved context of another process.
Process Identity	Process identity remains unchanged (same process continues execution in kernel space).	Process identity changes (CPU switches from Process P_A to Process P_B).

Parameter	Mode Switch	Context Switch
Overhead	Minimal overhead (saves minimal PC and register flags onto kernel stack).	Significant overhead (saves full PCB state, flushes CPU cache lines, re-loads Page Tables/TLB).
Triggering Cause	Hardware interrupt, software trap instruction, or system call invocation.	Time quantum expiration, higher-priority preemption, or process entering blocked state.

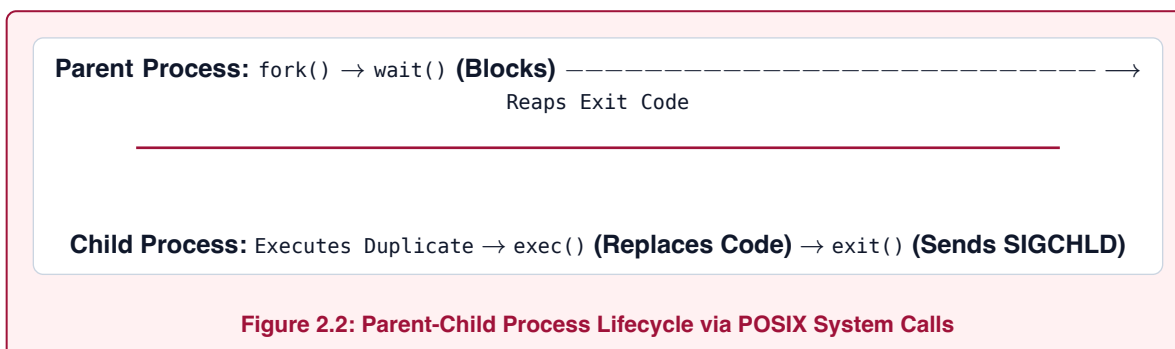
POSIX Process Management API and Lifecycle

In UNIX/Linux systems, process creation follows a distinct copy-and-replace lifecycle using C system calls.

1. **fork() System Call:**
 - Creates an exact duplicate child process of the caller parent process.
 - Child receives a duplicate copy of parent address space (optimized via **Copy-On-Write (COW)**).
 - **Return Values:** Returns 0 to the child process, returns child's PID to parent process, and -1 on failure.
2. **exec() Family System Calls:** Replaces the current process virtual memory space with a new executable binary program from disk.
3. **wait() / waitpid() System Calls:** Blocks parent execution until child process completes, retrieving child's exit status code.
4. **exit() System Call:** Terminates calling process, releases allocated RAM/files, and leaves exit status code in PCB.

Zombie vs. Orphan Processes

- **Zombie Process:** A process that has completed execution via `exit()`, but its entry remains in the kernel Process Table because its parent has not yet executed `wait()` to read its exit status code.
- **Orphan Process:** A child process whose parent process terminated before calling `wait()`. Inherited automatically by the `init` / `systemd` process (PID 1), which periodically calls `wait()` to reap them.



Source: Silberschatz, Ch 3, p. 126-145 | Source: Tanenbaum, Ch 2, p. 85-105

Threads and Multithreading

Thread Concept and Multithreaded Process Architecture

A **Thread** is the fundamental unit of CPU utilization. It represents an independent sequential stream of execution within a process.

1. **Shared Resources Across Threads in Process:** Memory address space (Text, Data, Heap segments), open file descriptors, child processes, signals, and environment settings.

2. **Per-Thread Private State:** Unique Thread ID (TID), Program Counter (PC), CPU Data Registers, and isolated Thread Execution Stack.

User-Level Threads (ULT) vs. Kernel-Level Threads (KLT)

Parameter	User-Level Threads (ULT)	Kernel-Level Threads (KLT)
Management Layer	Managed entirely by user-space run-time thread library (e.g., POSIX GNU Portable Threads).	Managed directly by OS kernel scheduler.
OS Kernel Visibility	Invisible to OS kernel; kernel sees only a single process.	Directly visible and scheduled individually by OS kernel.
Switching Overhead	Extremely fast (user-mode context switch without kernel intervention).	Slower (requires kernel mode switch and hardware state save).
Blocking I/O Effect	If one ULT executes a blocking system call, entire process blocks!	If one KLT blocks, kernel schedules another thread of the process.
Multiprocessing	Cannot utilize multiple physical CPU cores concurrently.	Scales seamlessly across multi-core symmetric multiprocessors.

Multithreading Mapping Models

1. **Many-to-One Model ($M : 1$):** Maps multiple user-level threads to a single kernel thread. Fast switching, but cannot use multi-core CPUs and blocks process on single thread I/O.
2. **One-to-One Model ($1 : 1$):** Maps each user thread directly to an individual kernel thread (Standard in Linux, Windows). Provides true multi-core concurrency.
3. **Many-to-Many Model ($M : N$):** Multiplexes M user threads onto N kernel threads ($M \geq N$). Achieves optimal balance of speed and multi-core scalability.

Source: Stallings, Ch 4, p. 155-180 | Source: Silberschatz, Ch 4, p. 161-185

CPU Scheduling Principles and Algorithms

Processor Scheduler Types and Scheduling Metrics

1. **Long-Term Scheduler (Job Scheduler):** Selects processes from disk queue and loads them into RAM ready queue. Controls **Degree of Multiprogramming**.
2. **Medium-Term Scheduler (Swapper):** Handles process swapping between main memory and secondary swap disk to adjust memory load.
3. **Short-Term Scheduler (CPU Scheduler):** Selects from RAM ready queue and assigns CPU core execution. Executes extremely frequently (every 10 – 100 ms).

Core Quantitative Scheduling Criteria

- **CPU Utilization:** Percentage of time CPU actively executes instructions (40% – 90%).
- **Throughput:** Number of completed processes per unit time.
- **Turnaround Time (T_T):** Interval from process arrival to complete termination ($T_T = T_C - T_A$).
- **Waiting Time (T_W):** Total time spent waiting inside ready queue ($T_W = T_T - T_B$).
- **Response Time (T_R):** Interval from process arrival to first CPU response.

Comprehensive Comparison of CPU Scheduling Algorithms

Algorithm	Preemptive?	Key Advantages	Major Drawbacks & Pitfalls
FCFS	Non-Preemptive	Simple, fair, zero scheduling overhead.	Convoy Effect: Short processes wait behind long CPU-bound process.
SJF	Non-Preemptive	Mathematically optimal minimum average waiting time.	Impossible to know exact next CPU burst time; Starvation of long jobs.
SRTF	Preemptive	Minimizes turnaround time for incoming short jobs.	Frequent context-switch overhead; starvation of long jobs.
Priority	Both supported	Assigns CPU based on critical task importance.	Indefinite Starvation: Resolved via Aging (gradually increasing priority).
Round Robin	Preemptive	Excellent response time for interactive systems.	Performance depends on quantum q (q too small $\rightarrow$ high overhead; q huge $\rightarrow$ FCFS).
MLFQ	Preemptive	Adaptive; separates CPU-bound & I/O-bound jobs.	Complex configuration of queues, demotion rules, and time slices.

Source: Stallings, Ch 9, p. 385-420 | Source: Silberschatz, Ch 6, p. 261-295

End of Operating Systems Unit 2 Theory Notes — SPPU TE IT 2024 Pattern